

Автономная некоммерческая организация высшего образования
«Университет Иннополис»

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)**
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

**FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)**
Academic Program
09.03.01 «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»

Field of Study:
«Computer Science»

Тема **Разработка системы мониторинга производительности и
обнаружения регрессий в Android-приложениях**

Topic **Development of a Performance Monitoring and Regression
Detection System for Android Applications**

Работу выполнил /
Prepared by

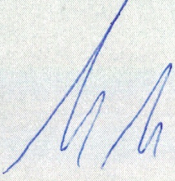
**Миннемуллин Наиль
Фидаилевич /
Nail Minnemullin**



подпись / signature

Руководитель
выпускной
квалификационной
работы / *Final
Qualification Work
Supervisor*

**Маццара Мануэль /
Manuel Mazzara**



подпись / signature

Иннополис, Innopolis, 2026

Contents

1	Introduction	7
1.1	Background and motivation	7
1.2	Research Gap	8
1.3	Research Questions and Aim	8
1.4	Novelty and Contribution	9
1.5	Structure	10
2	Literature Review	11
2.1	Performance monitoring	12
2.2	Approaches for Performance Data Collection	12
2.2.1	Pre-release Profiling	12
2.2.2	Post-release Monitoring	14
2.3	Performance Metrics	16
2.4	Performance Regression Detection	17
2.4.1	Approaches to Detecting Performance Regressions	18
2.4.2	Mobile-Specific Considerations and Research Gap	19
2.5	Summary	20
3	Methodology	22
3.1	System requirements and constraints	22

3.2	High-Level System Architecture	23
3.3	Mobile SDK Design	24
3.3.1	Metrics Selection and Collection	24
3.3.2	UI performance metrics	24
3.3.3	CPU usage	26
3.3.4	Memory usage	27
3.3.5	Data Aggregation and Transmission	27
3.4	Backend Design	28
3.4.1	Database Design	29
3.4.2	Data Partitioning and Cohorts	31
3.4.3	Performance Regression Detection	31
3.5	Summary	33
4	Implementation	34
4.1	Mobile SDK Implementation	34
4.1.1	Initialization and Lifecycle Management	35
4.1.2	Performance Metrics Collectors	36
4.1.3	Data Buffering and Network Transmission	39
4.2	Backend Implementation	41
4.2.1	User and Project Management	41
4.2.2	Data Ingestion	42
4.3	Performance Regression Detection	44
4.4	Frontend Implementation	46
4.4.1	User Authorization	46
4.4.2	Project Management	47
4.4.3	Metrics Explorer	48
4.4.4	Regressions	49

4.5	Summary	50
5	Evaluation and Discussion	51
5.1	Validation Setup	51
5.2	SDK Overhead Evaluation	52
5.3	End-to-End System Testing	54
5.3.1	Experiment Setup	55
5.3.2	Scenario 1: Frame-Time Regression	55
5.3.3	Scenario 2: No Code Change	56
5.3.4	Scenario 3: Regression on Low-Cohort Devices Only . . .	57
5.4	Discussion of Results	58
5.5	Limitations and Future Work	60
6	Conclusion	62
	Bibliography cited	64

List of Tables

I	Test Devices Used in the Evaluation	52
II	Resource Overhead per SDK	53
III	Cold-Start Overhead per SDK (mean of twenty launches)	53
IV	Frame-Time P95 Shift on Cheddar With No Code Change	57
V	Frame-Time P95 Shift Across Four Runs, Aggregated and Per Cohort	58

List of Figures

1	High-Level System Architecture	23
2	Sequence Diagram of SDK Data Collection and Transmission . .	28
3	Relational data model used by the backend	30
4	Login page of the dashboard	47
5	Project list page	47
6	Metrics Explorer: filter selection	48
7	Metrics Explorer: frame time chart with a detected regression . .	48
8	Regressions page of the dashboard	49
9	Regression alert delivered to a messaging app	50
10	Resource overhead per SDK. Left panel: mean CPU, PSS, and Java heap over the ninety-second scrolling session. Right panel: mean cold-start time over twenty launches.	54
11	Dashboard view of frame-time samples for the baseline build (left, dark blue) and the regression build (right, light blue) on the same screen. The regression build shows higher and more frequent frame-time spikes.	56

12	Recorded frame-time regression shown in the dashboard (a) and in the developer alert (b), each reporting the affected metric, screen, cohort, version pair, and P95 shift.	56
----	--	----

Abstract

Each new release of a mobile application can introduce a performance regression, for example through added features or updated dependencies. On Android these regressions are difficult to detect, because user devices vary widely in hardware and operating system, so the same release can run differently from one device to another. Regression detection is well established on the server side, but Android tools usually rely on fixed thresholds that a developer sets by hand and do not account for this device variety. This thesis presents a complete, open-source system that monitors Android applications and automatically detects performance regressions between releases. Its distinguishing element is that, instead of comparing all devices together, the system groups devices with similar capabilities into performance cohorts and compares each new release against the previous one within each cohort, so that a regression appearing only on weaker devices is not hidden among the stronger ones. The system has three parts: a mobile SDK on the device, a backend, and a frontend. Moreover, it sends an alert to developers when a regression is found. Because the SDK runs on the user's device, its overhead must remain low. Under a realistic workload it added 19.5 percentage points of CPU usage, 6.3 MB (3.9%) of memory, and 276 ms (15.9%) of cold-start time, around four times less CPU and nine times less memory than two widely used industry SDKs. The whole system was tested end to end, which it successfully passed. These results show that post-release monitoring and automatic cohort-aware regression detection can be applied to Android applications on real devices at a low cost to the user and with little setup for the developer.

Chapter 1

Introduction

1.1 Background and motivation

Mobile applications are now an essential part of everyday life, and users expect them to be fast, responsive, and stable. Poor performance (e.g. slow startup, high CPU usage, or freezing UI) can quickly degrade the user experience and result in negative reviews [1], [2]. Due to this reason, performance monitoring has become a crucial part of modern software development. Applications are also updated frequently, and any new release can unintentionally degrade performance on users' devices. This kind of release-to-release degradation, known as a performance regression, may stay invisible during development and appear only after the update reaches real users.

What makes Android performance monitoring particularly challenging is the wide variety of devices, operating system versions, and hardware configurations, a problem known as fragmentation [3]. The same application may perform differently across devices, so the most reliable way to understand its real-world behaviour is to observe it on real user devices under real usage conditions [4].

Monitoring performance in this way must address three challenges. First, because it runs on real user devices, it must add little runtime overhead, so that collecting metrics does not itself degrade the user experience. Second, it must account for the different performance levels of devices, so that a regression appearing only on lower-end devices is not hidden by stronger hardware when all devices are measured together. Third, it must detect regressions automatically, instead of relying on thresholds that a developer sets by hand for each metric and screen. Existing Android tools address some of these challenges individually but not all of them together.

1.2 Research Gap

Existing Android monitoring platforms provide metric collection, dashboards, and configurable alerting, but they require developers to define alert conditions by hand for individual metrics, screens, and use cases, and they do not group devices into performance cohorts. As a result, configuring and maintaining monitoring rules becomes time-consuming, and a regression that affects only one class of Android devices can stay hidden in the aggregated metrics. This creates a need for an automated regression detection approach that reduces manual alert configuration and explicitly accounts for device fragmentation. This gap is what the current work aims to address.

1.3 Research Questions and Aim

The object of study is the runtime performance of Android applications on real user devices, in particular how it changes between consecutive releases.

The aim of this thesis is to develop a system that collects performance metrics from running Android applications and automatically detects performance regressions introduced by new releases, while accounting for the different performance levels of user devices. It gathers key metrics directly from users' devices, sends the data to a backend, and notifies developers when a regression is detected.

The study addresses the following research questions:

1. How can Android application performance be monitored with low runtime overhead?
2. How to develop a system that automatically detects performance regressions introduced by new application releases?
3. Can performance cohorts help detect regressions that affect only specific classes of Android devices?

1.4 Novelty and Contribution

The main contribution of this thesis is a complete system for Android performance monitoring and regression detection. It covers the full pipeline, from on-device metric collection to developer alerts. The system is open source and already deployed¹, so developers can integrate the SDK into their own applications.

The novel element is that the regression comparison is stratified by device performance cohort. Each release is compared with the previous one within groups of devices of similar capability, so a regression that affects only weaker devices is detected instead of being averaged away.

¹<https://perfx.ru>

1.5 Structure

The thesis is organised as follows.

Chapter 2 reviews related work on mobile performance monitoring and regression detection. It compares pre-release profiling with post-release monitoring, looks at which performance metrics earlier studies relied on, and sets out the research gap behind this work.

Chapter 3 turns to the design of the proposed system. It covers the requirements, the overall architecture, the mobile SDK and the backend, and the regression detection methods that were weighed before one was chosen.

Chapter 4 describes how the system was built, from the mobile SDK and the backend to the frontend dashboard and the regression detection pipeline.

Chapter 5 evaluates the proposed system. It describes the validation setup, measures the overhead introduced by the SDK and evaluates the regression detection pipeline on synthetic regressions injected into a demo application, and discusses the results in relation to the research questions, together with their limitations.

Finally, Chapter 6 concludes the thesis by restating the problem and what was done, explaining the usefulness of the work, and outlining directions for future work.

Chapter 2

Literature Review

This chapter reviews existing research on mobile application profiling and monitoring. It looks at how performance metrics are collected and at which metrics matter most for mobile development. It then examines the main approaches to performance regression detection, which prepares the choice of method made later in this work.

The review is organised into three main parts. The first part compares the two main approaches to collecting performance metrics in mobile applications. The second part summarises the metrics used in previous research and explains how they contribute to the assessment of application performance and user experience. The third part reviews existing approaches to performance regression detection, from manual threshold-based alerting to release-to-release comparison in large-scale production systems. Together, these parts show the current state of research and provide a foundation for identifying the gap that the proposed system aims to address.

2.1 Performance monitoring

Performance strongly shapes how users experience a mobile application and whether they keep using it. Slow response times, high energy use, or a laggy interface frustrate users and often end in poor reviews and uninstalls [5]. Catching such problems early keeps an application usable and its users satisfied, as Tang *et al.* [1] demonstrate. A broad survey of mobile performance optimisation techniques confirms that these problems are pervasive across the field [6]. Two strategies are commonly used to understand application performance: profiling and monitoring. Profiling runs during development or testing and yields detailed, high-resolution data. Monitoring, in contrast, takes place after the application is released and runs on real user devices. It must introduce as little overhead as possible to prevent any interference with normal app usage.

The Android ecosystem adds complexity to performance evaluation due to fragmentation, the wide variety of hardware, operating system versions, and user behaviours [3], [7]. This diversity makes it difficult to reproduce rare or device-specific issues found in the wild. The same app may behave differently on various devices depending on their environment, OS version, and hardware specifications [4]. This problem motivates research into tools and methods for collecting and analysing mobile performance data effectively.

2.2 Approaches for Performance Data Collection

2.2.1 Pre-release Profiling

Several tools have been developed to support performance profiling during development. One example is ProfileDroid by Wei *et al.* [8], which uses a

multi-layer approach to profiling. It gathers data from four layers: application metadata, user-interface interaction traces, operating-system metrics such as CPU and memory usage, and network activity. Looking at these layers together lets developers see how the parts of an application interact, for instance how a single UI event triggers a CPU spike or a network request. The main strength of ProfileDroid is its ability to correlate performance across layers, giving a complete system-level view. However, its high overhead and dependence on a controlled environment limit its practical use on real devices.

Another relevant project is AndroStep [9], which focuses on analysing Android storage performance during I/O operations. The authors found that existing Android benchmarking tools did not simulate realistic app behaviour, especially missing operations like `fsync()` followed by 4 KB random writes - a common and essential operation used by SQLite, the embedded relational database built into Android for local data storage [9]. The tool is useful for benchmarking Android storage, but it measures nothing beyond storage and has to be run by hand, which limits it to a test environment rather than a live one.

PowDroid, introduced by Bouaffar *et al.* in 2021 [10], was designed to measure the energy consumption of applications. However, like most pre-release tools, it works only under controlled conditions, which makes it difficult to test applications on a large range of real devices.

In the field of UI performance, a new state-of-the-art work, GuiWatcher, was proposed by Liu *et al.* [11]. They presented an advanced way to detect UI lags and freezes. GuiWatcher uses computer vision techniques on screen recordings and is able to detect three types of problematic frames: "junky" frames (dropped or delayed ones), long loading frames (lengthy transition), and frozen frames. This method reflects what users actually see rather than relying only on timing

logs. However, it is computationally expensive and suitable only for use during development or continuous integration testing. It also raises privacy concerns, because it records the device screen.

ADEL, by Zhang *et al.* [12], targets network performance and energy leaks. It extends the platform to tag downloaded data and follow how that data moves through an application, which reveals unnecessary network activity that wastes energy. The tool is not built for production monitoring, however, because it modifies the Android system and adds high overhead, so it cannot run on real user devices.

At the platform level, Android provides a system tracing infrastructure [13] used by tools such as Perfetto [14], which record kernel-level events including CPU scheduling, memory pressure, and frame rendering timelines. These tools give detailed visibility into what the system is doing at any moment, but they require a developer-connected device and produce large volumes of data, so they are designed for diagnosing specific issues under controlled conditions rather than for ongoing collection from deployed applications.

2.2.2 Post-release Monitoring

The main idea of post-release monitoring is to collect metrics from apps as they run on real user devices [15]. The main advantage of this approach is that it captures realistic usage patterns and real-world diversity that are hard to reproduce in a controlled test environment. The main challenge is to minimise overhead to avoid affecting user experience.

One of the first studies in this area was AppInsight [16], which introduced a method for instrumenting applications without modifying their source code. The framework automatically injects instrumentation into the compiled bytecode and

collects runtime traces. By collecting traces from 30 users over several months, the authors were able to identify performance bottlenecks that were not visible during development. However, this approach is limited because developers cannot easily add custom metrics.

Another example is Hubble [17], which is currently integrated into all Huawei devices. The main feature of Hubble is that it monitors the performance in an efficient in-memory buffer, where older data is constantly overwritten [17]. The relevant metrics are only persisted when a performance issue is detected, providing engineers with detailed runtime traces from just before it occurred. This approach combines high efficiency with low storage cost.

PerfProbe [18] works across layers to find the root cause of a performance problem rather than only its symptom. It traces both the application and the system on real devices at low overhead, then uses statistical analysis to flag functions that behave abnormally and tie them to resource factors such as CPU, memory, or I/O. A developer thus learns not only where a slowdown happens but why. Across 22 Android apps, PerfProbe diagnosed real issues and reduced user-perceived latency in six of them by 32% to 86% [18].

Panappticon [19] is a lightweight tracing system that records performance across the application, system, and kernel layers. It traces critical execution paths automatically by linking each user input to the display update it produces. Unlike AppInsight, it can also catch problems that arise from interactions between several apps or system components. Over a one-month study its overhead stayed around 6%, low enough for ongoing use in the field.

2.3 Performance Metrics

Performance is not captured by a single number but by a set of metrics, which fall into two kinds: those that describe how an application uses device resources, and those that describe what the user actually perceives. Resource metrics explain where a performance problem originates, while user-perceived metrics show how strongly it affects the user. The two are linked, because resource pressure usually surfaces as a slower or less responsive interface.

On the resource side, the most common metrics are CPU usage, memory usage, storage I/O, and energy consumption. High CPU utilisation overloads threads and reduces responsiveness, and CPU contention has been shown to be a frequent cause of poor user experience on Android [20]. Memory usage matters for stability: an application that consumes too much RAM can trigger the system's low-memory killer or frequent garbage-collection pauses, so developers profile heap usage to catch allocation bottlenecks and memory leaks [21]. Storage I/O also affects latency, mostly on devices with slower flash memory, where a slow read can make a screen take noticeably longer to open [9]. Energy use is just as visible. Surveys put battery life at the top of users' performance concerns, and people avoid applications they consider too power-hungry [22], so finding energy leaks is an important part of mobile optimisation.

On the user-perceived side, several more specific metrics describe responsiveness. Frame rendering time is the most direct of these. A frame has to be drawn within the display's refresh budget, roughly 16.7 ms on a 60 Hz screen, and a frame that misses this budget shows up as stutter or a dropped frame [23]. Frames per second (FPS) carries the same information in a different form and follows directly from frame time. Startup time, the wait before an application becomes usable

after launch, largely sets the user's first impression [23]. Interaction latency is the delay between a user action and the first visible response. Even when CPU and memory look normal, a long delay here noticeably reduces satisfaction, and human-computer interaction research treats a delay of over one second as the point where the user's flow of thought is interrupted [24]. The most severe responsiveness failure is an Application Not Responding (ANR) event, raised when the main thread stays blocked for several seconds, which often makes the user abandon the application [23], [25].

In practice no single metric is sufficient, and tools combine several to build a fuller picture. ProfileDroid and Panappticon correlate UI and system activity, while PerfProbe links CPU and I/O data with execution paths. The choice of metrics depends on the goal: pre-release profiling collects many low-level metrics, whereas post-release monitoring favours a smaller set of user-perceived metrics, such as frame time, startup time, and interaction latency, that are cheap to measure on real devices and reflect more directly what the user actually experiences.

2.4 Performance Regression Detection

A performance regression is a situation where software still works correctly but performs worse than a previous baseline, for example slower response times or higher resource usage. In mobile applications this usually appears after an update, as increased frame rendering time, slower startup, higher CPU usage, or greater memory consumption. Regression detection aims to identify a sustained degradation in performance across a population of users between two releases, rather than isolated, short-lived spikes.

2.4.1 Approaches to Detecting Performance Regressions

The simplest and most widely used approach in industry is threshold-based alerting. Tools such as Firebase Performance Monitoring [2] and Google Play Vitals [26] let developers set fixed limits on individual metrics, for example a maximum startup time, and raise an alert when a limit is exceeded. This approach is easy to understand, but the thresholds must be set manually for each metric and application, and a single fixed value rarely fits every device and usage pattern, which leads to false alerts or missed regressions. Such tools also react to absolute values rather than to changes between versions, so a slowdown introduced by a new release goes unnoticed as long as it stays within the configured limit.

A more adaptive approach compares current performance against a historical baseline instead of a fixed limit. Nguyen *et al.* [27] build control limits from a baseline run and flag a new run when it deviates too far from them, which removes the need to choose absolute thresholds by hand. Shang *et al.* [28] take a related approach and use regression models trained on clustered performance counters to reduce the manual overhead of selecting which metrics to watch. Both methods, however, were designed for controlled load-test runs of server software rather than for data collected from real user devices over time. On mobile, Carat [29] applies a similar idea at the population level: it collects energy data from many users, builds a baseline of normal behaviour for each application, and reports devices and apps that deviate from it. Carat compares devices rather than consecutive releases, so it can locate an abnormal app but does not attribute a regression to a particular version. Industry observability platforms such as Datadog go further and compare live metrics against an automatically learned historical or seasonal baseline, flagging statistically significant deviations [30]. These tools react to anomalies as they unfold over time rather than to the difference between two releases, and they target

server and infrastructure metrics rather than mobile applications on user devices.

The approach closest to the needs of this work compares each new release directly against the previous one, so that a detected regression is attributed to a specific version. This is the principle behind canary analysis, which is widely used in industry to gate deployments by comparing a new version against a stable baseline before full rollout [31], [32]. At large scale, Meta’s FBDetect [33] monitors hundreds of thousands of time series in production, compares versions to catch even small regressions, and filters out false alerts caused by transient issues such as load spikes or rolling updates. These systems show that release-to-release comparison is an effective and interpretable way to detect regressions, because each alert is tied to a concrete change. Both systems run inside the data centre, where the software executes on controlled and largely homogeneous servers. Prior work on detecting performance regressions in heterogeneous server environments [34] has shown that comparing results across different hardware configurations without stratification leads to false alerts, and on Android, where device diversity is far greater, this problem is more acute. These systems therefore do not deal with device fragmentation and have not been applied to performance data collected from heterogeneous Android devices after release.

2.4.2 Mobile-Specific Considerations and Research Gap

Applying these ideas to Android raises additional constraints. Devices differ widely in hardware and operating system version, so the same release can perform differently across devices, and comparing them directly can make normal hardware differences look like regressions. Devices are also resource-constrained, and collecting detailed traces from real users raises privacy concerns. A practical design therefore keeps the on-device part lightweight and moves the heavier anal-

ysis to a server. A recent study, MobileUPReg [35], addresses a related problem by comparing user-perceived performance metrics across OS versions rather than across application releases, and it shows that OS updates can introduce regressions in frame delivery and interaction time that go undetected by conventional testing. Its focus on OS-level changes is complementary to, but distinct from, the present work, which detects regressions introduced by new versions of the application itself.

Existing Android monitoring platforms provide metric collection, dashboards, and configurable alerting, but they require developers to define alert conditions by hand for individual metrics, traces, screens, and use cases, and they do not group devices into performance cohorts. As a result, configuring and maintaining monitoring rules becomes time-consuming, and a regression that affects only one class of Android devices can stay hidden in the aggregated metrics. This creates a need for an automated post-release regression detection approach that reduces manual alert configuration and explicitly accounts for device fragmentation. The proposed system addresses this gap by comparing consecutive releases on the server with a single relative threshold and grouping the comparison by device performance cohort.

2.5 Summary

This chapter reviewed existing research on mobile performance profiling, monitoring, and regression detection. Pre-release tools such as ProfileDroid, AndroStep and PowDroid offer deep system-level insights but introduce high overhead and require controlled environments, making them unsuitable for real user devices. Post-release monitoring systems such as AppInsight, PerfProbe, and Hubble avoid

these drawbacks by gathering data straight from real users at low overhead.

The chapter also separated two kinds of metric: resource use such as CPU, memory, storage I/O, and energy, and the user-perceived responsiveness that those resources ultimately affect. It closed with regression detection, moving from manual threshold-based alerting to baseline and release-to-release comparison in large-scale systems such as FBDetect. The recurring finding is that comparing a new release against the one before it is an effective way to catch sustained degradation.

Even so, no existing solution combines low-overhead post-release monitoring on Android with cohort-stratified regression detection. This gap is what the proposed system sets out to fill.

Chapter 3

Methodology

This chapter describes the design of the proposed performance monitoring system. Section 3.1 lists the requirements that influenced the design choices. Section 3.2 gives a high-level overview of the architecture and shows how the components work together. Sections 3.3, 3.4 describe the details of the mobile SDK and the backend. Section 3.4.3 discusses the regression detection methods that were considered and highlights the selected one.

3.1 System requirements and constraints

Since the SDK (Software Development Kit) runs on real user devices, it must be lightweight so that it does not affect the user experience of the host application. Moreover, the system must be adaptable to different device performance levels. Finally, regression detection must flag sustained degradation between consecutive releases rather than reacting to momentary spikes.

3.2 High-Level System Architecture

The system consists of four components: the mobile SDK (client), the backend, the frontend, and the regression detection engine. Figure 1 shows how they are connected.

The mobile SDK is a library that developers integrate into their Android applications. It collects performance metrics on users' devices and sends them to the backend for further analysis.

Metrics from user devices arrive at the backend, which stores them efficiently and serves them to the frontend through REST APIs. The backend is also where user authentication and project management happen.

The frontend is a web application for developers. Through it they manage their projects, explore the collected metrics, and look over any regressions that were found. It pulls all of this from the backend through the same REST APIs and lays the data out in a readable form.

The regression detection engine analyses the incoming metrics to find signs of performance degradation. If such a regression is detected, it can notify developers in a messaging app.

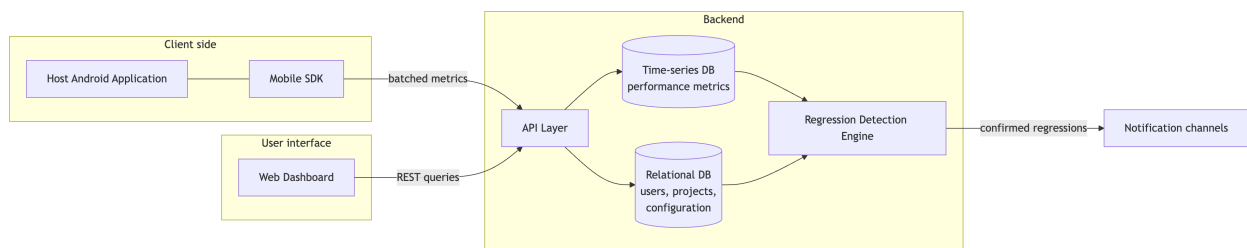


Fig. 1. High-Level System Architecture

3.3 Mobile SDK Design

3.3.1 Metrics Selection and Collection

The metrics were selected based on the literature review in Chapter 2. The guiding criterion was that the collected metrics should reflect how the user actually perceives the application, rather than only describe low-level system behaviour. As discussed in the review, performance metrics fall into two kinds: resource metrics, such as CPU and memory usage, that explain where a problem originates, and user-perceived metrics, such as frame time, startup time, and interaction delay, that show how strongly that problem affects the user. The two are linked, because resource pressure usually surfaces as a slower or less responsive interface. The SDK therefore collects user-perceived metrics as the primary signal of experienced performance, together with resource metrics that help explain the cause of a perceived slowdown.

These metrics are collected in two ways: as periodic samples and as discrete events. Periodic samples, such as CPU usage and frame time, are taken at regular intervals. Discrete events, such as crashes, ANRs, and app startups, are recorded immediately when they happen. This approach allows the system to capture both the ongoing performance of the application and the rare but critical events that strongly affect user experience.

3.3.2 UI performance metrics

For UI performance, the SDK tracks frame time, FPS, startup time, interaction delays, and ANR events.

Frame time is one of the most important metrics for measuring UI re-

sponsiveness. It represents the time needed to render a single frame of the user interface. The target frame time depends on the display refresh rate of the device. For example, on a 60 Hz display, the frame time should stay below approximately 16.67 ms in order to keep animations and interactions smooth [23].

Frame time is measured by recording the timestamps of two consecutive frames and calculating the difference between them. If t_i and t_{i-1} are the timestamps of two consecutive frames, the frame time is calculated as follows:

$$FrameTime(ms) = \frac{t_i - t_{i-1}}{10^6} \quad (1)$$

Frames per second (FPS) is a related metric that is derived directly from the frame time. It is calculated as:

$$FPS = \frac{1000}{FrameTime(ms)} \quad (2)$$

User interaction delay measures the time between a user action (e.g. tap on a screen or a swipe) and the first visible response of the application. It is obtained by recording the timestamp of the user event and the timestamp of the first frame that was rendered after the event was processed. The delay is then calculated as:

$$InteractionDelay(ms) = t_{\text{response}} - t_{\text{event}} \quad (3)$$

Startup time shows how long the application takes to become visually responsive after it has been launched. It is defined as the time difference between the moment the application starts and the moment when the first fully rendered frame is drawn on the screen.

ANR (Application Not Responding) events happen when the application

is unable to handle user input for a long period of time. This is a critical metric, because ANRs directly affect user experience and can lead to the application being terminated by the operating system. ANR events are detected by monitoring the main thread. If the main thread remains blocked for more than a predefined threshold (for example, 5 seconds), an ANR event is recorded.

3.3.3 CPU usage

CPU usage is measured from the CPU time consumed by the application process. The process CPU time is read at the start and end of each sampling interval, and the usage is the increase in process CPU time divided by the elapsed wall-clock time of the interval. A system-wide alternative, which would divide the process CPU time by the total CPU time of all processes, would give a value normalised to the device capacity, but it requires reading `/proc/stat`, which modern Android versions restrict for security reasons. The process-level measure is used instead, as it relies only on the public platform API and adds little overhead. Because the application can run on several cores at once, this value can exceed 100%, and it depends on the core count and clock speed of the device. Cross-device comparison is therefore not made by the formula itself, but by the cohort stratification and the release-pair comparison described in Section 3.4.3. The method may also miss short spikes that occur between two samples, but this is acceptable here, because the goal is to detect sustained degradation rather than momentary load.

Let $\Delta CPU_{process}$ be the increase in process CPU time and Δt_{wall} the elapsed wall-clock time over the same interval. The CPU usage of the application is

calculated as:

$$CPU\% = \frac{\Delta CPU_{process}}{\Delta t_{wall}} \cdot 100 \quad (4)$$

3.3.4 Memory usage

There are also several methods for measuring the memory used by an Android application. In this work, memory usage is measured using the Proportional Set Size (PSS) of the application process. The PSS value represents how much physical memory is effectively used by the application, including a proportional share of the memory that is shared with other processes.

The memory usage is reported in megabytes and is calculated from the PSS value in kilobytes as:

$$MemoryUsage(MB) = \frac{PSS(KB)}{1024} \quad (5)$$

This metric is important, because it helps to detect memory leaks and excessive memory consumption, which are frequent causes of application slowdowns and of termination by the operating system.

3.3.5 Data Aggregation and Transmission

The metrics collected on the device are not sent immediately. Instead, they are first stored in a local buffer and then transmitted in batches. Batching was chosen because the radio interface stays in a high-power state for several seconds after each transmission, so frequent network calls are disproportionately energy-intensive [36], [37]. Sending metrics in batches reduces this cost and also lowers the load on the backend.

A batch is sent when one of two conditions is satisfied: either the number of metrics in the buffer reaches a predefined value N , or the time since the last transmission exceeds a time window W .

Figure 2 shows the sequence of operations performed by the SDK during data collection and transmission.

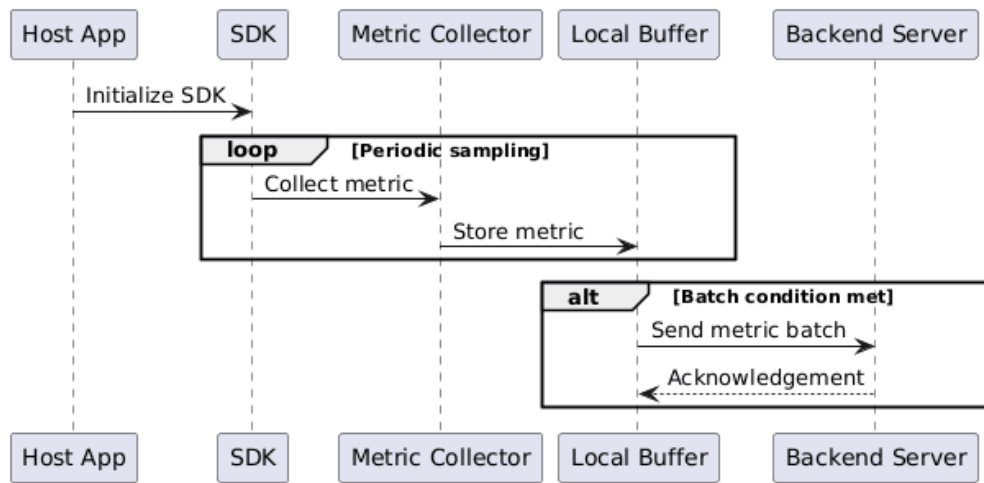


Fig. 2. Sequence Diagram of SDK Data Collection and Transmission

3.4 Backend Design

The backend has four jobs: it receives metrics from the devices, authenticates users, exposes APIs for the frontend, and runs the regression detection algorithms.

When a batch of metrics arrives from the SDK, the backend validates the request, enriches each record with device cohort information, and writes the data into the time-series database. After that, the frontend can query the stored metrics through REST endpoints, and the regression detection engine runs in the background to analyse new data as soon as it arrives.

3.4.1 Database Design

The backend relies on two kinds of database. A time-series database stores the performance metrics, and a relational database stores the rest: the users, their projects, and the regressions found for those projects. The time-series store was chosen because it handles large volumes of time-stamped data efficiently, whereas the relational store fits the structured, interconnected records that describe accounts and their projects.

The relational side holds four entities, shown in Figure 3. A *user* owns one or more *projects*, each identified by its Android package name. A project, in turn, owns a catalogue of *version releases* and its detected *regressions*. The *version releases* entity records each application version observed for a project, together with when it was first seen and how many samples it has accumulated. This catalogue is refreshed periodically from the time-series store and is used to list the releases of a project. The *regressions* entity records each confirmed regression: the affected metric, screen, and device cohort, the baseline and current versions with their P95 values, and a status that marks it as open or closed. This ownership structure also enforces tenancy isolation, since every query can be scoped to the projects that belong to the requesting user.

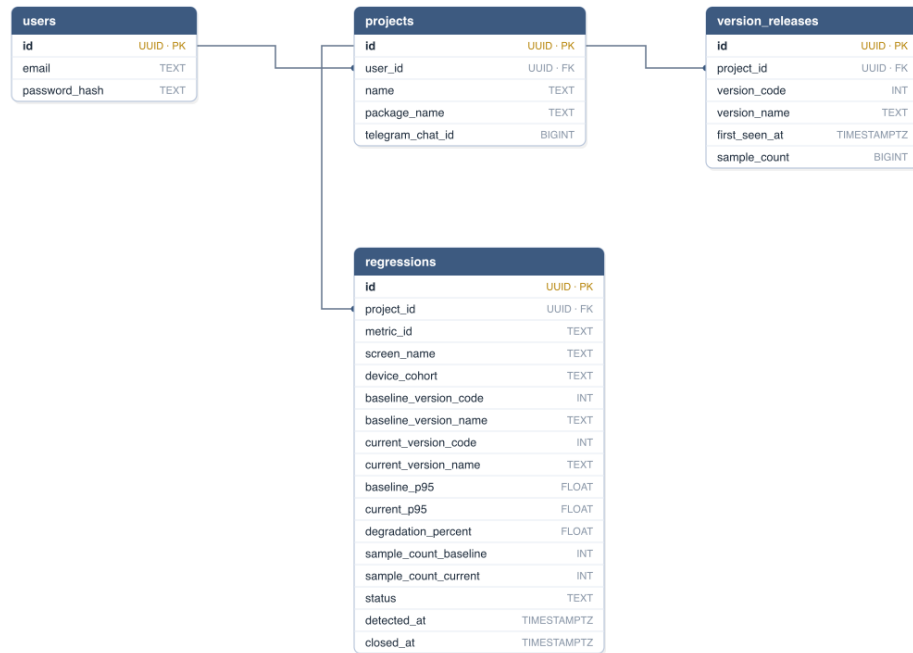


Fig. 3. Relational data model used by the backend

Performance metrics are kept separately in the time-series store and are not modelled as relational entities. For this purpose the system uses ClickHouse, a column-oriented database. A column-oriented layout fits the metric workload well: ingestion is append-only and write-heavy, while analysis reads only a few columns at a time and aggregates them over large ranges, such as computing a percentile per version [38]. Each metric record carries a value, its timestamp, and the dimensions along which it is later grouped, namely the project, the metric type, the screen, the device cohort, and the application version. The data is organised so that a query restricted to a recent period and to a single project and metric scans only the relevant subset, which keeps the regression-detection queries efficient as the volume of data grows.

Metrics are also retained only for a limited period of time. Older data is removed automatically, which keeps the working set relevant and reduces storage

cost. The physical layout of both stores, including table definitions and indexing, is described in Chapter 4.

3.4.2 Data Partitioning and Cohorts

Before applying any detection model, the incoming data must be grouped into meaningful subsets. Comparing high-end devices directly with low-end devices could easily produce false positives, because the natural hardware limits of a low-end device might be interpreted as a software regression. To avoid this, the data is partitioned along three dimensions: the metric type (for example, CPU usage or frame time), the application screen, and the device performance cohort (Low, Medium, or High). The detection algorithms are then executed independently on each subset.

3.4.3 Performance Regression Detection

The regression detection pipeline compares consecutive application releases using a release-pair P95 shift. Samples are grouped by version code rather than by a fixed time window, and each release is compared against the one that precedes it. Fixed absolute thresholds were rejected because each one must be calibrated by hand for every metric and application, and a fixed limit does not move as baseline performance shifts over time. A mean-based comparison was avoided as well, since the mean is easily pulled by outliers and copes badly with the heavy-tailed, non-normal distributions that frame time and startup time usually follow. Heavier methods such as ARIMA forecasting or machine-learning models were ruled out too, because they need labelled training data or careful tuning that a lightweight production system cannot justify.

Detection runs only on user-perceived metrics, such as frame time and startup time, because a regression matters when the user actually feels it. Startup time, the delay before the application becomes responsive after launch, is treated as a regression target alongside frame time. Resource metrics such as CPU and memory usage are still collected, but they serve to explain the cause of a perceived slowdown rather than to raise alerts, so a fluctuation in resource use that the user never notices does not trigger a false alarm.

A version is only eligible for comparison after it has accumulated a minimum number of samples. This maturity gate prevents comparisons based on a handful of early measurements, where a single noisy observation could shift the P95 far enough to cross the threshold and produce a false alert.

For each mature version group, the 95th percentile (P_{95}) of the metric is computed. A regression is reported when the relative increase from the baseline version to the current version exceeds a configurable threshold:

$$\frac{P_{95}(\text{current}) - P_{95}(\text{baseline})}{P_{95}(\text{baseline})} > \text{Threshold} \quad (6)$$

Using P_{95} rather than the mean focuses detection on the worst user experiences rather than the average, and makes the result robust to skewed distributions [27], [39]. The threshold expresses a directly interpretable criterion: a given percentage of degradation. Unlike the absolute, per-metric limits used by the threshold-based tools discussed in Chapter 2, it is a relative value that compares each release against the previous one, so a single default (15%) applies across all metrics, screens, and device cohorts. The developer therefore does not have to choose or calibrate a threshold for every metric and application. The system uses this default out of the box. This avoids the main drawback of fixed absolute thresholds, where each limit

must be set and maintained by hand. The pipeline runs on production data without any labelled examples, and its core comparison is expressed as a single aggregation query, making it computationally efficient and suitable for ongoing execution. The detection rule is deliberately kept simple and interpretable: a single comparison against a configurable percentage.

Because P_{95} is taken over the whole version group, it ignores anything that affects fewer than five percent of the samples. A momentary spike is a handful of high values among the thousands collected from many users over the lifetime of a version, so it sits above the 95th percentile and does not move P_{95} . The value shifts only when the degradation becomes frequent enough to reach the top five percent of the samples, which is a sustained problem rather than a spike. This is what lets the rule flag sustained degradation while ignoring momentary spikes.

3.5 Summary

This chapter presented the methodology behind the proposed performance monitoring system. The design was driven by the requirements from the literature review, with an emphasis on low overhead, energy efficiency, and robustness across devices. The mobile SDK collects metrics on the user's device and sends them to the backend in batches. The backend stores them, exposes them through an API, and runs rule-based regression detection on user cohorts. Together, these parts give developers automated insight into how their applications perform in production. The next chapter turns this design into a working system and describes the technical problems that came up along the way.

Chapter 4

Implementation

This chapter describes how the design from Chapter 3 was turned into working software.¹ Section 4.1 covers the mobile SDK, from initialization to the individual collectors and the buffering and transmission pipeline. Section 4.2 describes the backend, focusing on user management and the ingestion path. Section 4.3 explains the regression detection pipeline in detail. Section 4.4 then walks through the frontend web application and its main features.

4.1 Mobile SDK Implementation

The mobile part of the monitoring system was implemented as an Android library written in Kotlin. Kotlin was chosen because it is the official, recommended language for Android development [40].

The SDK is distributed through a Maven repository, which allows developers to add it to their project easily by adding a dependency to their `build.gradle` file.

¹The source code is available at <https://github.com/nailka/PerfX>.

4.1.1 Initialization and Lifecycle Management

One of the requirements for the SDK was to make the integration as simple as possible for developers. To achieve this, the entire setup is done with a single call to the `PerfX.initialize()` method inside the application class. The developer only needs to provide the application context and a unique project ID.

Initialization runs a short sequence of steps. The `AppInfoProvider` collects device and app metadata (OS version, device model, app version). The SDK then opens the local Room database (`MetricDatabase`), starts the background sync manager, and creates the metric collectors. Finally, the SDK registers an `Application.ActivityLifecycleCallbacks` listener so it can follow the active screen and pause network sync while the app sits in the background, which saves battery.

Listing 4.1: SDK Initialization

```
1 fun initialize(application: Application, projectId: String) {  
2     if (isRunning) return  
3  
4     appInfo = AppInfoProvider().get(application, projectId)  
5     db = MetricDatabase.getInstance(application)  
6     SyncManager.startPeriodicSync(application)  
7  
8     // ... Collector initialization (described below)  
9  
10    registerActivityCallback(application)  
11    isRunning = true  
12 }
```

4.1.2 Performance Metrics Collectors

Each metric collector extends a common abstract class called `PerformanceCollector`. This class defines a single `collect()` method that returns a Kotlin Flow of Metric objects, which allows the SDK to treat all collectors in a uniform way.

Listing 4.2: PerformanceCollector Abstract Class

```
1 internal abstract class PerformanceCollector {  
2     abstract fun collect(intervalMs: Long): Flow<Metric>  
3     abstract fun stop()  
4 }
```

All individual metric streams are merged into a single Flow inside the `MetricsRepository` class. This design makes it easy to add new collectors in the future without changing the rest of the pipeline.

1. Memory Collector

The `RamCollector` runs on a background thread and measures the memory footprint of the application at a fixed sampling interval. It uses two Android APIs: `Debug.getMemoryInfo()` to get the total PSS (Proportional Set Size) of the process, and the Java Runtime to calculate the current Java heap usage. The Java heap usage is calculated as the difference between the total allocated heap and the currently free heap. This value is what gets emitted as the metric, because it reflects the memory actually used by the application code.

Listing 4.3: Memory Collector Implementation

```
1 val memoryInfo = Debug.MemoryInfo()  
2 Debug.getMemoryInfo(memoryInfo)  
3  
4 val runtime = Runtime.getRuntime()  
5 val javaHeapUsedMb = (runtime.totalMemory() - runtime.freeMemory()  
    ) / (1024.0 * 1024.0)
```

```

6  val pssMb = memoryInfo.totalPss / 1024.0
7
8  emit (Metric.MemoryUsage(timestamp = System.currentTimeMillis(),
    value = javaHeapUsedMb))

```

2. CPU Collector

Because modern Android versions restrict direct access to the `/proc/stat` file for security reasons, the `CpuCollector` uses the `android.os.Process` API instead. From this API it reads the accumulated CPU time of the application process at two points, one sampling interval apart. The CPU percentage is the CPU time used during that interval divided by the wall-clock length of the interval:

$$CPU\% = \frac{\Delta CPU_{process}}{\Delta t_{wall}} \cdot 100 \quad (7)$$

This ratio expresses how much CPU time the application used relative to real time, so the values stay comparable across devices with different core counts.

3. Frame Time Collector

Frame rendering time is captured through a `Choreographer.FrameCallback` registered on the main thread. Android invokes this callback just before each new frame is drawn and passes the frame timestamp in nanoseconds. Subtracting the timestamps of two consecutive callbacks gives the raw frame time. Individual frame times are accumulated over a fixed interval and then averaged before being emitted as a single metric. This aggregation reduces the number of data points that need to be stored and transmitted, while still capturing the overall rendering performance of the window.

Listing 4.4: Frame Time Collector: core measurement

```

1  val callback = Choreographer.FrameCallback { frameTimeNs ->
2      if (lastFrameTimeNs != 0L) {

```

```
3         val deltaMs = (frameTimeNs - lastFrameTimeNs) / 1_000_000
4         .0
5         if (deltaMs >= 0.0) {
6             frameCount++
7             totalFrameTimeMs += deltaMs
8         }
9         lastFrameTimeNs = frameTimeNs
10        Choreographer.getInstance().postFrameCallback(frameCallback)
11    }
12
13    // Emitted every intervalMs
14    val avgFrameTime = if (frameCount > 0) totalFrameTimeMs /
15        frameCount else 0.0
16    emit(Metric.FrameTime(timestamp = System.currentTimeMillis(),
17        value = avgFrameTime))
```

4. Startup Time Collector

Startup time is measured by recording the timestamp at the moment the application process starts and comparing it to the timestamp of the first fully drawn frame. The SDK uses the `reportFullyDrawn()` API to detect when the first screen has finished rendering. The difference between these two timestamps is reported as the startup time.

5. Interaction Delay Collector

User interaction delay is measured by intercepting touch events on the root view of each activity. When a touch event is received, its timestamp is recorded. The delay is then calculated as the time between the touch event and the first frame rendered after the event was processed, which is detected using the `Choreographer` callback.

6. ANR Detector

The SDK detects ANR events using a watchdog pattern. A background thread posts a short task to the main thread's Handler and waits for it. If the task does not run within the timeout (5 seconds by default, matching the official Android documentation [23]), the watchdog treats the main thread as blocked and records an ANR. The approach needs no special permissions and works on every Android version.

Listing 4.5: ANR Watchdog: detection logic

```
1 val mainHandler = Handler(Looper.getMainLooper())
2 var mainThreadResponded = false
3
4 mainHandler.post { mainThreadResponded = true }
5 Thread.sleep(ANR_TIMEOUT_MS)
6
7 if (!mainThreadResponded) {
8     emit(Metric.Anr(timestamp = System.currentTimeMillis()))
9 }
```

4.1.3 Data Buffering and Network Transmission

The SDK uses a two-tier buffering approach to make sure that no metrics are lost if the application crashes or the network is temporarily unavailable.

Tier 1: In-Memory Buffer

As metrics arrive from the various collectors, they are first stored in a `ConcurrentLinkedQueue`. This structure was chosen because several collectors write at the same time from different background threads, and a plain list could lead to race conditions. A `ConcurrentLinkedQueue` handles concurrent access safely without explicit locks, which keeps the overhead low.

When the number of items in the memory buffer reaches a predefined limit (MEMORY_LIMIT), the buffer is flushed to disk.

Listing 4.6: In-Memory Buffering

```
1 flow.collect { metric ->
2     memoryBuffer.add(metric)
3     memoryBufferSize += 1
4
5     if (memoryBufferSize >= Constants.MEMORY_LIMIT) {
6         flushToDisk(db, context)
7         memoryBufferSize = 0
8     }
9 }
```

Tier 2: Disk Storage and Network Synchronization

On a flush, the metrics move to a local SQLite database managed by the Room library. Each record is stored together with the name of the screen that was visible at the time, which the lifecycle callback registered during initialization keeps track of.

Once the database holds DISK_LIMIT records, the SDK starts a network synchronization immediately. A background job also runs every 15 minutes and uploads whatever is left, even if that limit has not been reached, so metrics are still sent when the application is used only rarely.

The network layer uses Retrofit and OkHttp. The SDK reads a batch of records from the database, serializes it to JSON with Kotlinx Serialization, and sends it to the backend in a single HTTP POST request. On success, the uploaded records are deleted from the local database. On failure, they stay on disk and are retried at the next synchronization.

Listing 4.7: Network Synchronization

```
1 val batchApi = BatchApi(  
2     appInfo = PerfX.appInfo ,  
3     metrics = batch.map { it.mapToApi() }  
4 )  
5  
6 val response = NetworkClient.api.uploadBatch(batchApi)  
7  
8 if (response.isSuccessful) {  
9     db.metricDao().deleteMetrics(batch.map { it.id })  
10 }
```

4.2 Backend Implementation

The backend was built in Kotlin with the Ktor framework. Ktor is lightweight and asynchronous, which fits a REST API well. It serves each request on a coroutine, so it can handle many connections at once without blocking threads.

It relies on two databases: PostgreSQL for relational data such as user accounts and project metadata, and ClickHouse for the time-series performance metrics.

4.2.1 User and Project Management

PostgreSQL holds the user accounts, projects, version releases, and detected regressions, following the relational data model shown in Chapter 3 (Figure 3).

Passwords are hashed with BCrypt before they are stored. At login, the server checks the supplied password against the stored hash, and on success it issues a JSON Web Token (JWT). Every later request to the API must carry this

token, which keeps each user's data accessible only to that user.

To create a project, a developer submits a name and an Android package name, and the server rejects either one if it is already taken. Once the project passes this check, it is saved and linked to the developer's account. A background job periodically refreshes the `version_releases` catalogue from ClickHouse, recording each observed version code together with its first-seen time and accumulated sample count. The dashboard uses this catalogue to list the releases of a project.

4.2.2 Data Ingestion

The data ingestion endpoint is the most performance-critical part of the backend, because the mobile SDKs from many users can send large numbers of metrics at the same time. Storing this data in PostgreSQL would not be efficient enough for this workload. For this reason, all performance metrics are stored in ClickHouse, which is a column-oriented database optimized for fast batch insertions and analytical range queries over time [38].

The `metric_records` table was designed to support the queries needed by the regression detection pipeline efficiently. The table uses the MergeTree engine, which is optimized for fast insertions and range scans. The data is partitioned by date so that queries over a recent time window only scan the relevant partitions. The primary sort key is `(project_id, metric_id, ts)`, which makes range queries over time for a specific project and metric very fast.

Listing 4.8: ClickHouse metric_records table schema

```
1 CREATE TABLE IF NOT EXISTS metrics.metric_records
2 (
3     project_id      String ,
4     package_name    String ,
5     ts              DateTime64(3, 'UTC') ,
6     metric_id       LowCardinality(String) ,
7     screen_name     LowCardinality(String) ,
8     value           Float64 ,
9     app_version     LowCardinality(String) ,
10    os_version       LowCardinality(String) ,
11    device_model     LowCardinality(String) ,
12    device_cohort    LowCardinality(String) DEFAULT 'Unknown' ,
13    received_at      DateTime DEFAULT now()
14 )
15 ENGINE = MergeTree
16 PARTITION BY toDate(ts)
17 ORDER BY (project_id , metric_id , ts);
```

When the mobile SDK sends a POST request to the `/ingest` endpoint, the server receives a batch of metrics together with the application and device metadata. Instead of inserting each metric individually, the `ClickHouseClient` formats the entire batch as a single `JSONEachRow` payload and executes one bulk INSERT query. Before saving each record, the server also assigns the device cohort (Low, Medium, or High) based on the hardware metadata sent by the SDK. Batching the inserts this way keeps pressure on the database low, so the server can absorb many simultaneous requests before it turns into a bottleneck.

4.3 Performance Regression Detection

The regression detection pipeline described in Chapter 3 was split into three stages: device cohort classification on the Ktor backend, per-version P95 aggregation using ClickHouse queries, and release-pair comparison in a Python service.

The first stage assigns the device cohort and runs on the backend. The SDK includes the total RAM and the CPU core count of the device in every batch it uploads. From these values the backend determines the device cohort. It reads this metadata from each incoming request, derives the cohort, and stores it with every metric before writing the data to ClickHouse.

Listing 4.9: Device cohort classification

```
1 fun calculateDeviceCohort(totalRamGb: Double, cpuCores: Int):  
    String {  
2     return when {  
3         totalRamGb <= 3.0 -> "Low"  
4         totalRamGb <= 6.0 && cpuCores <= 8 -> "Medium"  
5         else -> "High"  
6     }  
7 }
```

In the second stage, the pipeline computes the P95 of each version group inside ClickHouse with the built-in quantile function. The query groups records by project, metric, screen, device cohort, and version code, and discards any version group that has not yet collected enough samples to count as mature. The query returns a single P95 per (metric, screen, cohort, version) combination, ordered by version code so that consecutive releases can be paired up.

Listing 4.10: ClickHouse query for per-version P95 values

```
1 SELECT
2     project_id ,
3     metric_id ,
4     screen_name ,
5     device_cohort ,
6     version_code ,
7     version_name ,
8     quantile(0.95)(value) AS p95 ,
9     count()              AS cnt
10 FROM metric_records
11 GROUP BY
12     project_id , metric_id , screen_name ,
13     device_cohort , version_code , version_name
14 HAVING cnt >= {min_samples}
15 ORDER BY
16     project_id , metric_id , screen_name ,
17     device_cohort , version_code
```

In the final stage, once ClickHouse has returned the mature versions, the detector iterates over each group and compares consecutive version pairs. For every pair, it computes the relative P95 shift and evaluates it against the configured threshold. If the shift exceeds the threshold, the corresponding regression is recorded in the regressions table in PostgreSQL.

Listing 4.11: Release-pair comparison in the detection service

```
1 for i in range(len(versions) - 1):
2     baseline_vc , baseline_vn , baseline_p95 , baseline_cnt =
        versions[i]
3     current_vc , current_vn , current_p95 , current_cnt =
        versions[i + 1]
```

```
4
5     if baseline_p95 <= 0:
6         continue
7
8     delta = (current_p95 - baseline_p95) / baseline_p95
9
10    if delta > threshold:
11        upsert_regression(pg_conn, ..., delta)
```

The threshold is a single global value of 15%, defined in the detector configuration and applied uniformly to every group. The system does not store per-project or per-metric thresholds. Using one relative threshold avoids the manual per-metric calibration required by the absolute threshold-based tools discussed in Chapter 2.

Once a regression has been flagged, it is written to the `regressions` table in PostgreSQL together with the metric type, affected screen, device cohort, the baseline and current P95 values, and the degradation percentage. Detected regressions appear on the dashboard, where developers can inspect and investigate active issues.

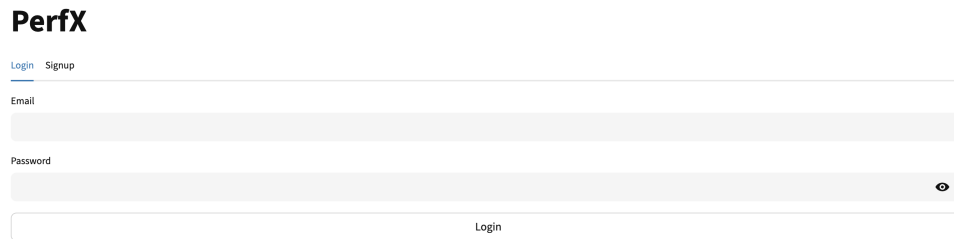
4.4 Frontend Implementation

The frontend was built with Streamlit, a Python framework for interactive web apps. Streamlit made it possible to build a data-driven interface quickly, without maintaining a separate frontend codebase. Plotly renders the charts.

4.4.1 User Authorization

Users sign up and log in with an email and password. The backend returns a JWT on a successful login, and the dashboard keeps it in the browser session.

From then on, every call to the backend API carries that token as proof of identity.

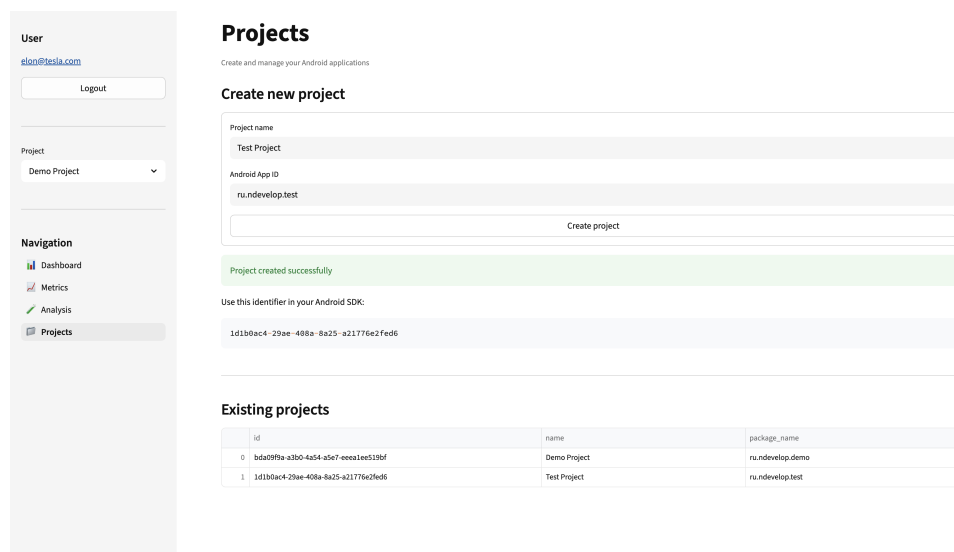


The image shows the login page of the PerfX dashboard. It features a header with the 'PerfX' logo and links for 'Login' and 'Signup'. Below the header, there are input fields for 'Email' and 'Password'. The 'Password' field has a toggle icon for visibility. At the bottom, there is a 'Login' button.

Fig. 4. Login page of the dashboard

4.4.2 Project Management

Once logged in, a user sees their existing projects and can add or remove projects from the same page. When a new project is created, the dashboard displays the unique project ID that the developer needs to use when initializing the mobile SDK in their Android application.



The image shows the project list page of the PerfX dashboard. It features a sidebar with a 'User' section (email: elon@tesla.com, Logout button) and a 'Navigation' section (Dashboard, Metrics, Analysis, Projects). The main content area is titled 'Projects' and includes a 'Create new project' form. The form has fields for 'Project name' (Test Project) and 'Android App ID' (ru.ndevelop.test), with a 'Create project' button. Below the form, a green message states 'Project created successfully'. A section titled 'Use this identifier in your Android SDK' displays the ID '1d1b0ac4-29ae-488a-8a25-a21776e2fed6'. At the bottom, a table titled 'Existing projects' lists the projects.

	id	name	package_name
0	bda09f9e-a3b0-4a54-a5e7-e0ea1ee513bf	Demo Project	ru.ndevelop.demo
1	1d1b0ac4-29ae-488a-8a25-a21776e2fed6	Test Project	ru.ndevelop.test

Fig. 5. Project list page

4.4.3 Metrics Explorer

The metrics explorer presents the raw data in a table, where each measurement appears with its timestamp, value, and metadata. The table can be exported to CSV for deeper analysis in tools such as Excel or Jupyter Notebook. This helps with debugging and with one-off analyses.

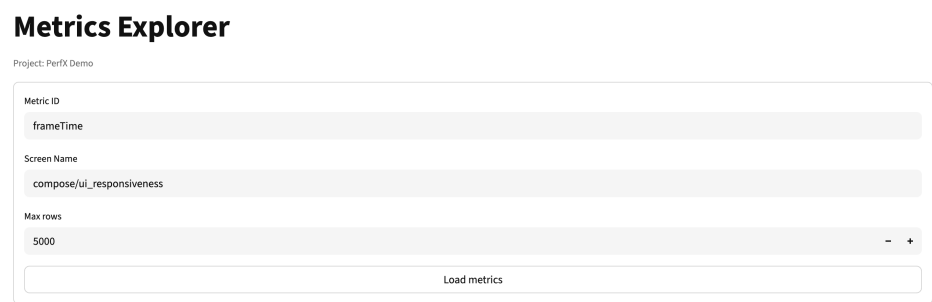


Fig. 6. Metrics Explorer: filter selection

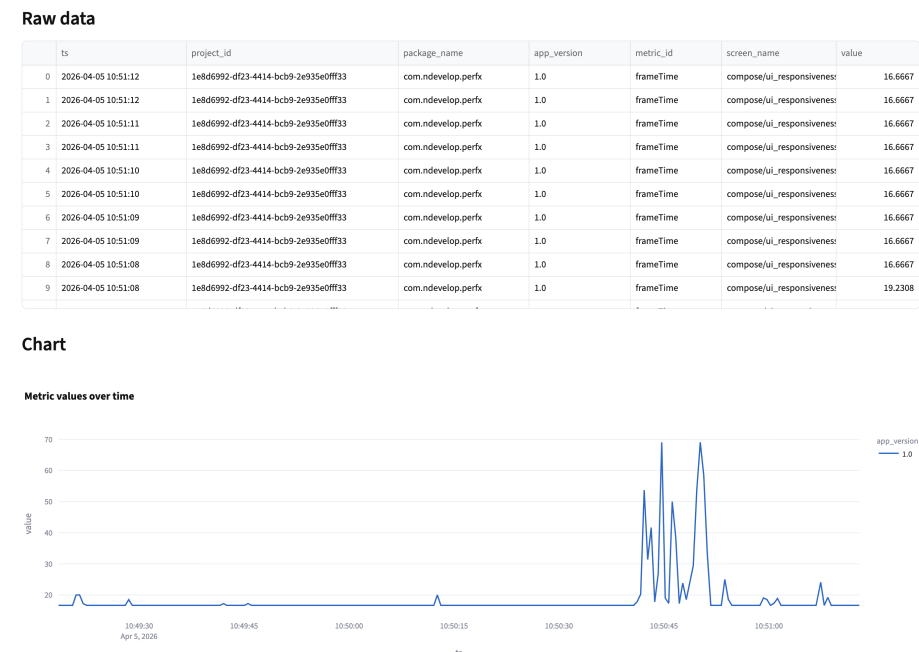


Fig. 7. Metrics Explorer: frame time chart with a detected regression

4.4.4 Regressions

The regressions page lists every regression that the detection pipeline has reported for the selected project. Each row shows the affected metric, screen, and device cohort, the version pair that was compared, the degradation, and whether the regression is open or closed. A summary at the top of the page counts how many regressions are open and how many are closed. From the same page a developer can filter the list and close a regression once it has been handled.

User
demo@perfx.ru
Logout

Project
PerFX Demo

Navigation
Metrics
Regressions
Projects

Filters
Status: all
Severity: Choose options
Metric: All
Version pair: All

Regressions

Total: 2 Open: 1 Closed: 1

Regression details

			Metric	Screen	Tier	Version pair	Degradation %	Status	Detected
0	🔴	🔥	frameTime	compose/ui_responsiveness	Medium	v2.0 → v3.0	262.2%	open	13 Jun 2026
1	🔴	✅	frameTime	compose/ui_responsiveness	Medium	v1.0 → v2.0	69.8%	closed	13 Jun 2026

Update regression status

Select regression
🔥 frameTime | compose/ui_responsiveness | Medium | v2.0 → v3.0

New status
open

Apply

Fig. 8. Regressions page of the dashboard

Developers can also receive regression alerts in messenger. Each message reports the affected metric, screen, and device cohort, the compared version pair, and the magnitude of the degradation.

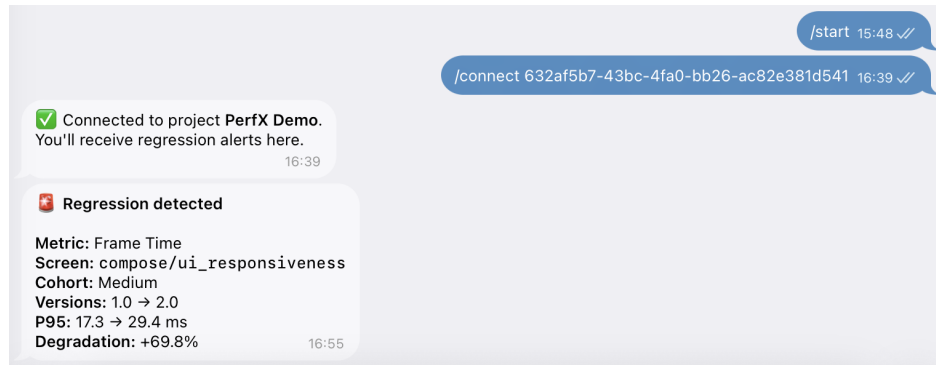


Fig. 9. Regression alert delivered to a messaging app

4.5 Summary

This chapter described the implementation of all four system components. The mobile SDK was built as a Kotlin Android library that collects six performance metrics, buffers the data in a two-tier storage layer, and transmits it to the backend in batches. The backend receives the data and stores it in two separate databases: ClickHouse for time-series metrics and PostgreSQL for user and project metadata. The regression detection pipeline was implemented as a Python service that computes per-version P95 values using ClickHouse aggregation queries and flags regressions when the relative P95 shift between consecutive releases exceeds the configured threshold. Finally, the frontend was built with Streamlit and lets developers explore the collected metrics and review detected regressions. The next chapter evaluates how well this system works in practice.

Chapter 5

Evaluation and Discussion

This chapter evaluates the system against the research questions stated in Chapter 1. Section 5.1 describes the validation setup. Section 5.2 measures the SDK overhead, addressing the first research question about low-overhead monitoring. Section 5.3 presents an end-to-end test of the whole system, addressing the second and third research questions on automatic regression detection and the role of device performance cohorts in detection. Section 5.4 summarises the findings, and Section 5.5 discusses the limitations.

5.1 Validation Setup

The experiments were carried out on one physical Android device and three emulator configurations, each defined by a different amount of RAM and a different number of CPU cores, so that the results cover all three performance cohorts defined in Section 3.4.3. Table I lists each device configuration together with its hardware characteristics, its Android version, and the cohort assigned by the backend. Individual scenarios may run more than one emulator instance of a

given configuration, and the per-scenario counts are reported in the corresponding subsection.

TABLE I
Test Devices Used in the Evaluation

Device	RAM (GB)	CPU cores	Android version	Cohort
Emulator (Low cohort)	2	2	11	Low
Emulator (Medium cohort)	4	4	13	Medium
Xiaomi Redmi Note 9 Pro	6	8	15	Medium
Emulator (High cohort)	8	4	14	High

Two separate applications were used across the evaluation. The overhead evaluation used Cheddar [41], an open-source Android client for the Hacker News website, representing a realistic workload of network requests, list rendering, and in-app navigation. The end-to-end system test used a dedicated demo application developed as part of this project, with a screen whose per-frame CPU work can be scaled to inject a cohort-sensitive frame-time regression. Two builds with different version codes were compared so the detector treated them as consecutive releases.

5.2 SDK Overhead Evaluation

The SDK overhead was measured under a single Cheddar workload, built four times under distinct product flavors. The baseline build (no monitoring SDK) defines the reference. PerfX is compared against two widely used industry SDKs: Firebase Performance Monitoring [2] and Datadog RUM. Each build was installed in turn, cold-started twenty times to measure startup time, then left running for ninety seconds while an automated scrolling script exercised the story feed and CPU and PSS memory were sampled every second.

TABLE II
Resource Overhead per SDK

Flavor	Avg CPU (%)	CPU overhead (pp)	Avg PSS (MB)	PSS overhead (MB)
Baseline	24.1	–	162.7	–
PerfX	43.6	+19.5	169.0	+6.3 (+3.9%)
Firebase	101.7	+77.6	222.1	+59.4 (+36.5%)
Datadog	102.5	+78.4	220.4	+57.7 (+35.5%)

TABLE III
Cold-Start Overhead per SDK (mean of twenty launches)

Flavor	Cold start (ms)	Overhead (ms)	Overhead (%)
Baseline	1737	–	–
PerfX	2013	+276	+15.9
Firebase	1945	+208	+12.0
Datadog	1914	+177	+10.2

PerfX added 19.5 percentage points of CPU usage, 6.3 MB (+3.9%) of resident memory, and 276 ms (+15.9%) of cold-start time. Firebase Performance Monitoring and Datadog RUM behaved similarly to each other, but produced overhead an order of magnitude higher: roughly 78 percentage points of CPU and around 58 MB of memory, between nine and ten times the PerfX memory cost and around four times the PerfX CPU cost. The cold-start penalties of the three SDKs were closer to one another, at 177–276 ms, since cold-start time is dominated by one-shot class loading and SDK initialisation rather than steady-state work.

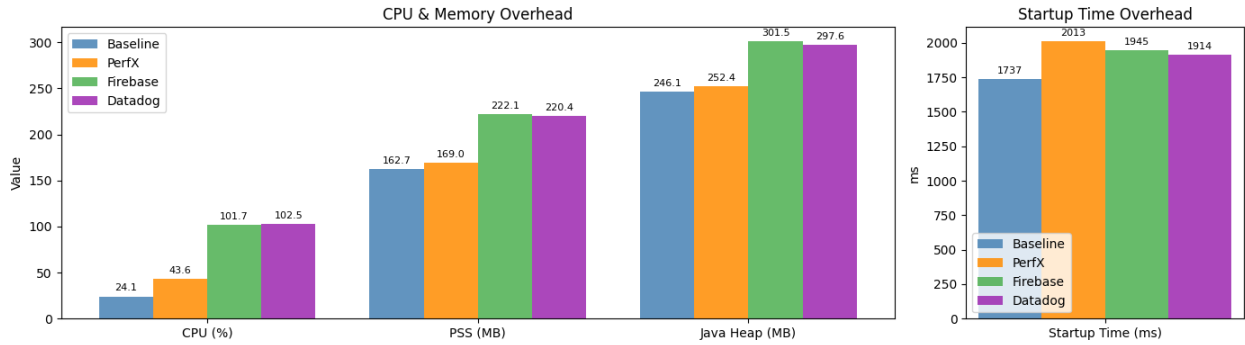


Fig. 10. Resource overhead per SDK. Left panel: mean CPU, PSS, and Java heap over the ninety-second scrolling session. Right panel: mean cold-start time over twenty launches.

The PerfX CPU cost remains the lowest of the three SDKs compared here, and is comparable to the approximately 6% reported for the Panappticon tracing system [19] once Cheddar’s higher baseline activity is taken into account. Together with the small steady-state memory overhead and a moderate one-time cold-start cost, these results support the low-overhead requirement defined in Section 3.1.

5.3 End-to-End System Testing

This section tests the full system end to end. A synthetic regression is injected into a build by adding extra CPU work per rendered frame, which raises frame time under load. The build then runs on real devices, and the full system, from the SDK through the backend and detector to the dashboard, is checked for the expected outcome. The detection rule is a deterministic threshold that flags a release when the relative P95 shift exceeds 15%, so the goal here is to verify that the system applies the rule correctly, not to estimate a statistical detection accuracy.

The system is tested in three scenarios. The first injects a frame-time regression and follows it through the whole system. The second compares two builds with no code change and checks that no alert is raised. The third runs a cohort-localised

regression on all cohorts and checks that the alert reaches only the affected one. The reported results describe the observed behaviour of the system on a controlled set of cases, not statistical estimates from production traffic.

5.3.1 Experiment Setup

Each scenario compared two builds under consecutive version codes, so the detector treated them as successive releases. The complete path was checked: the SDK collects the metric on the device, the backend ingests it, the detector compares the P95 of the two releases per metric, screen, and cohort, creates a regression record when the relative shift exceeds 15%, and sends an alert. The regression scenarios use the demo application, and the no-change scenario uses Cheddar. Each build was exercised for the same fixed run duration in every scenario, so the two versions in every release pair contributed similarly sized samples to the detector, and every version satisfied the minimum sample-count gate.

5.3.2 Scenario 1: Frame-Time Regression

The first scenario introduces a frame-time regression and checks whether the system works correctly. The regression adds a fixed amount of CPU work to each rendered frame, which raises the frame-time metric. The SDK collected the frame-time samples on the physical device, the backend deployed on a remote server received them, and the dashboard plotted the two builds together. Figure 11 shows the dashboard view, where the regression build reaches higher frame times more often than the baseline. The detector then compared the two releases and recorded a regression, which the dashboard lists on its regressions page in Figure 12a. The system then delivered the alert in Figure 12b, which reports the affected metric,

screen, cohort, version pair, and the resulting P95 shift. Each step produced the expected result.

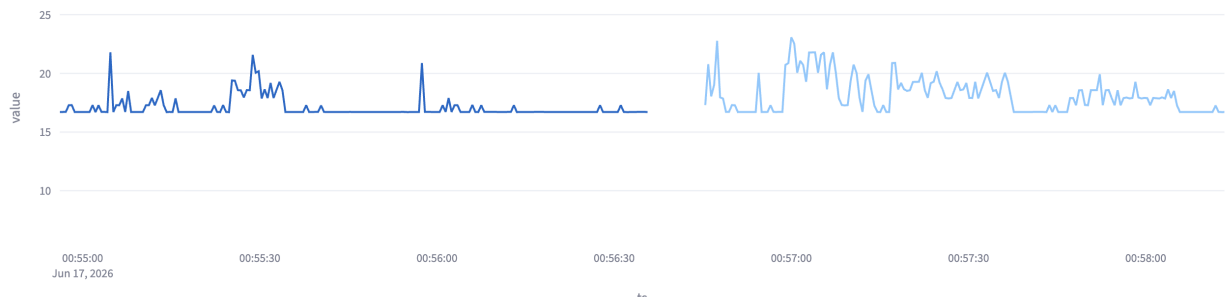


Fig. 11. Dashboard view of frame-time samples for the baseline build (left, dark blue) and the regression build (right, light blue) on the same screen. The regression build shows higher and more frequent frame-time spikes.

Regression details

			Metric	Screen	Tier	Version pair	Baseline P95	Current
0	🟡	🔥	frameTime	compose/ui_responsiveness	Medium	v7.0 → v8.0	22.78	

Update regression status

Select regression

🔥 frameTime | compose/ui_responsiveness | Medium | v7.0 → v8.0

New status

open

Apply

Regression detected
Metric: Frame Time
Screen: compose/ui_responsiveness
Cohort: Medium
Versions: 7.0 → 8.0
P95: 22.8 → 27.2 ms
Degradation: +19.6%
03:55

(a) Regressions page of the dashboard.

(b) Alert delivered to the developer.

Fig. 12. Recorded frame-time regression shown in the dashboard (a) and in the developer alert (b), each reporting the affected metric, screen, cohort, version pair, and P95 shift.

5.3.3 Scenario 2: No Code Change

The second scenario compares two consecutive builds with no code change, so any reported shift comes from measurement noise alone. Four builds of Cheddar

that differ only in version code were each scrolled through the story feed for three minutes, and the frame-time P95 shift between them was recorded. Cheddar was used because it gives a realistic workload of network requests and list rendering.

Table IV reports the frame-time P95 shift for the four runs, and Figure ?? plots the per-sample values of one run, where the two P95 lines almost coincide. No run crossed the 15 % threshold, so the detector created no regression record and sent no alert.

TABLE IV
Frame-Time P95 Shift on Cheddar With No Code Change

Pair	Baseline version P95 (ms)	Control version P95 (ms)	P95 shift (%)
1	17.82	18.64	4.60
2	18.14	17.91	-1.27
3	17.76	18.93	6.59
4	18.03	18.71	3.77
5	17.95	18.42	2.62
6	18.27	19.04	4.21
7	17.88	17.56	-1.79
8	18.11	19.09	5.41

5.3.4 Scenario 3: Regression on Low-Cohort Devices Only

The third scenario runs a frame-time regression that only affects performance on low-tier devices. The scenario used three High-cohort emulators, three Medium-cohort emulators, and one Low-cohort emulator, so the Low cohort contributed fewer samples than the other two. The collected samples were analysed in two ways: first aggregated across all devices without cohort stratification, and then split by device cohort. The experiment was repeated four times to confirm that the outcome was stable. Table V reports both the aggregated shift and the per-cohort shifts for each run.

TABLE V
Frame-Time P95 Shift Across Four Runs, Aggregated and Per Cohort

Run	Aggregated shift (%)	Per-cohort shift (%)		
		Low	Medium	High
1	8.4	27.0	9.1	5.2
2	7.6	25.8	8.4	4.7
3	9.2	28.3	9.8	5.9
4	8.1	26.4	8.7	5.0

Without cohort stratification, the aggregated shift stayed below the 15% threshold in every run, so the detector raised no alert. With cohort stratification, the Low cohort crossed the threshold in every run and triggered an alert, while the Medium and High cohorts stayed below it. The same samples therefore produced an alert only when analysed by cohort.

This matches the real device population, where most active Android devices fall into the Medium and High cohorts and the Low cohort is a minority [42]. When the same samples are analysed without cohort stratification, the strong shift on the Low cohort is diluted by the larger Medium and High groups, and the aggregated shift stays below the threshold. Cohort stratification recovers this signal by comparing each cohort separately, so a regression confined to the minority Low cohort still produces an alert.

5.4 Discussion of Results

This section interprets the results in relation to the three research questions defined in Chapter 1.

The first research question asks how Android application performance can be monitored with low runtime overhead. The results in Section 5.2 show that PerfX added 19.5 percentage points of CPU usage, 6.3 MB (+3.9%) of resident memory,

and 276 ms (+15.9%) of cold-start time under a realistic Cheddar workload. The same scenario produced 77.6 percentage points of CPU and 59.4 MB of memory for Firebase Performance Monitoring, and 78.4 percentage points and 57.7 MB for Datadog RUM, so PerfX uses about four times less CPU and roughly nine times less memory than either industry SDK while collecting a comparable set of metrics. These costs are consistent with the background sampling, two-tier buffering, and batched transmission design described in Chapter 4.

The second research question asks how to develop a system that automatically detects performance regressions introduced by new application releases. The pipeline groups metric samples by version code, metric, screen, and device cohort, applies a maturity gate, and then flags a regression when the relative P95 shift between two consecutive releases exceeds the fixed 15% threshold. This approach requires no manual labelling and no per-metric threshold tuning, and it runs against incoming production data.

The third research question asks whether performance cohorts help detect regressions that affect only specific classes of Android devices. The injected regression degraded frame time only on weaker hardware. Compared within cohorts, it was flagged on the Low cohort and correctly left the Medium and High cohorts untouched. If all devices had been compared together, the shift on the Low devices would have been diluted by the unaffected Medium and High devices and could have fallen below the threshold, so the regression would have been missed. Cohort stratification therefore allows a regression confined to one device class to be detected and attributed to that class. On the realistic Cheddar workload with no code change, the relative shift stayed below 9 % on every metric, so the system raised no false alert. Because the detection rule is a deterministic threshold, this is a verification that the system applies it correctly.

5.5 Limitations and Future Work

The evaluation has several limitations.

First, the system was not deployed to real users. Overhead and detection were measured on a small fixed set of devices running one usage scenario, which does not capture the variety of real-world usage.

Second, emulators do not fully replicate the hardware behaviour of physical devices, so the emulator results for CPU and memory should be treated as indicative rather than conclusive.

Third, the device cohort is assigned using a simple rule based only on total RAM and CPU core count. Two devices with the same RAM and the same number of cores can still differ in real performance because of their chipset, GPU, or thermal behaviour, so this rule may place a device in the wrong cohort and weaken the cohort-based comparison.

Fourth, the detector is designed to identify release-to-release regressions. As a consequence, gradual performance degradation accumulated over multiple releases may remain undetected if no individual release exceeds the configured threshold.

These limitations point to several directions for future work. The most important next step is a real-world deployment on a live application with real users, which would test the system against naturally occurring regressions under varied hardware, usage patterns, and network conditions. The cohort classification could also be improved. Instead of relying on static hardware attributes such as RAM and CPU core count, cohorts could be derived from each device's observed performance, for example by grouping devices according to a lightweight on-device benchmark. Finally, the set of collected metrics could be extended with battery

and network usage, which are also relevant to mobile user experience [22] but are not currently collected.

Chapter 6

Conclusion

This thesis studied performance monitoring and regression detection for Android applications after release. The main problem is that existing Android monitoring platforms require developers to define alert conditions by hand for individual metrics, screens, and use cases, and they do not group devices into performance cohorts, so monitoring rules become time-consuming to maintain and a regression that affects only one class of devices can stay hidden in the aggregated metrics.

To solve this, a complete system was designed and built. A mobile SDK collects performance metrics on real user devices with low overhead and sends them to the backend in batches. A Ktor backend stores the data in ClickHouse and PostgreSQL and assigns a device cohort to each record. A detection pipeline then compares two consecutive releases by the relative P95 shift, grouped by metric, screen, and cohort, and informs developers when a new release makes performance worse. The system was evaluated for runtime overhead and through an end-to-end test of its regression detection. These results, together with the answers to the research questions, are presented in Chapter 5.

The main contribution of this work is a complete, open-source system that covers the full pipeline, from on-device metric collection to developer alerts. Its novel element is that the release-to-release comparison is stratified by device performance cohort. Android devices differ widely in capability, so the comparison is run separately within groups of devices of similar capability, and a regression that appears only on weaker devices is still detected instead of being averaged away.

Bibliography cited

- [1] Y. Tang, H. Wang, X. Zhan, *et al.*, “A systematical study on application performance management libraries for apps,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 3044–3065, Aug. 2022, ISSN: 0098-5589. DOI: 10.1109/TSE.2021.3077654.
- [2] Firebase. “Firebase performance monitoring.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://firebase.google.com/docs/perf-mon>.
- [3] M. Kamran, J. Rashid, and M. Nisar, “Android fragmentation classification, causes, problems and solutions,” *International Journal of Computer Network and Information Security*, vol. 14, pp. 992–999, Sep. 2016.
- [4] Z. Dong, Y. Zhao, T. Liu, *et al.*, “Same app, different behaviors: Uncovering device-specific behaviors in android apps,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE ’24, Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 2099–2109, ISBN: 9798400712487. DOI: 10.1145/3691620.3695272.
- [5] H. Khalid, E. Shihab, M. Nagappan, and A. E. Hassan, “What do mobile app users complain about?” *IEEE Software*, vol. 32, no. 3, pp. 70–77, 2015. DOI: 10.1109/MS.2014.50.

- [6] M. Hort *et al.*, “A survey of performance optimization for mobile applications,” *Journal of Systems and Software*, 2021, Online version consulted 30 Oct 2025.
- [7] X. Sun, X. Chen, Y. Liu, J. Grundy, and L. Li, “Taming android fragmentation through lightweight crowdsourced testing,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 6, pp. 3599–3615, Jun. 2023, ISSN: 0098-5589.
- [8] X. Wei, L. Gomez, I. Neamtii, and M. Faloutsos, “Profiledroid: Multi-layer profiling of android applications,” Aug. 2012. DOI: 10.1145 / 2348543 . 2348563.
- [9] S. Jeong, K. Lee, J.-Y. Hwang, S. Lee, and Y. Won, “Androstep: Android storage performance analysis tool,” in *Software Engineering*, 2013.
- [10] F. Bouaffar, O. L. Goaer, and A. Nouredine, “Powdroid: Energy profiling of android applications,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 2021, pp. 251–254. DOI: 10.1109/ASEW52652.2021.00055.
- [11] W. Liu, F. Lin, L. Guo, T.-H. Chen, and A. Hassan, “Guiwatcher: Automatically detecting gui lags by analyzing mobile application screencasts,” Feb. 2025. DOI: 10.48550/arXiv.2502.04202.
- [12] L. Zhang, M. Gordon, R. Dick, Z. Mao, P. Dinda, and L. Yang, “Adel: An automatic detector of energy leaks for smartphone applications,” Oct. 2012, pp. 363–372. DOI: 10.1145/2380445.2380503.
- [13] Android Developers. “Overview of system tracing.” Accessed: 20 Oct 2025. (2025), [Online]. Available: <https://developer.android.com/topic/performance/tracing>.

- [14] Google Perfetto Team. “Perfetto: System profiling, app tracing and trace analysis.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://perfetto.dev/docs/>.
- [15] Android Developers. “Monitoring app performance.” Accessed: 30 Oct 2025. (2022), [Online]. Available: <https://medium.com/androiddevelopers/monitoring-app-performance-ebf9af4ebe36>.
- [16] L. R. Sivalingam, J. Padhye, S. Agarwal, R. Mahajan, I. Obermiller, and S. Shayandeh, “Appinsight: Mobile app performance monitoring in the wild,” Nov. 2012.
- [17] Y. Luo, K. Rodrigues, C. Li, *et al.*, “Hubble: Performance debugging with In-Production, Just-In-Time method tracing on android,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, Carlsbad, CA: USENIX Association, Jul. 2022, pp. 787–803, ISBN: 978-1-939133-28-1.
- [18] D. K. Hong, A. Nikraves, Z. M. Mao, M. Ketkar, and M. Kishinevsky, “Perfprobe: A systematic, cross-layer performance diagnosis framework for mobile platforms,” in *2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 2019, pp. 50–61. DOI: 10.1109/MOBILESoft.2019.00018.
- [19] L. Zhang, D. Bild, R. Dick, Z. Mao, and P. Dinda, “Panappticon: Event-based tracing to measure mobile application and platform performance,” Sep. 2013, pp. 1–10. DOI: 10.1109/CODES-ISSS.2013.6659020.
- [20] J. Fu, Y. Wang, Y. Zhou, and X. Wang, “How resource utilization influences ui responsiveness of android software,” *Information and Software Technology*, vol. 141, p. 106 728, Sep. 2021. DOI: 10.1016/j.infsof.2021.106728.

- [21] B. Li, Q. Zhao, S. Jiao, and X. Liu, “Droidperf: Profiling memory objects on android devices,” Jul. 2023, pp. 1–15. DOI: 10.1145/3570361.3592503.
- [22] R. Rua and J. Saraiva, “A large-scale empirical study on mobile performance: Energy, run-time and memory,” *Empirical Softw. Engg.*, vol. 29, no. 1, Dec. 2023, ISSN: 1382-3256. DOI: 10.1007/s10664-023-10391-y.
- [23] Android Developers. “Android vitals: App startup time, slow rendering, and ANRs.” Accessed: 19 April 2026. (2026), [Online]. Available: <https://developer.android.com/topic/performance/vitals>.
- [24] J. Nielsen. “Response times: The 3 important limits.” Accessed: 2026-06-15. (1993), [Online]. Available: <https://www.nngroup.com/articles/response-times-3-important-limits/>.
- [25] Z. Li, Y. Liu, C. Liu, *et al.*, “Experience: Aging or Glitching? Why does Android stop responding and what can we do about it?” In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom '20)*, ACM, 2020, pp. 1–14. DOI: 10.1145/3372224.3380897.
- [26] Google Play Console. “Monitor your app’s technical quality with android vitals.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://play.google.com/console/about/vitals/>.
- [27] T. H. D. Nguyen, B. Adams, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora, “Automated detection of performance regressions using statistical process control techniques,” in *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering (ICPE '12)*, ACM, 2012, pp. 299–310.

- [28] W. Shang, A. E. Hassan, M. Nasser, and P. Flora, “Automated detection of performance regressions using regression models on clustered performance counters,” in *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*, ACM, 2015, pp. 15–26.
- [29] A. Oliner, A. Iyer, I. Stoica, E. Lagerspetz, and S. Tarkoma, “Carat: Collaborative energy diagnosis for mobile devices,” Nov. 2013. DOI: 10.1145/2517351.2517354.
- [30] Datadog. “Anomaly monitor.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://docs.datadoghq.com/monitors/types/anomaly/>.
- [31] Netflix Technology Blog. “Automated canary analysis at Netflix with Kayenta.” (2018), [Online]. Available: <https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69> (visited on 06/09/2026).
- [32] M. Lindon, C. Sanden, and V. Shirikian, “Rapid regression detection in software deployments through sequential testing,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ACM, 2022, pp. 3336–3346. DOI: 10.1145/3534678.3539099.
- [33] D. Y. Yoon, Y. Wang, M. Yu, *et al.*, “FBDetect: Catching tiny performance regressions at hyperscale through in-production monitoring,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP ’24)*, ACM, 2024.
- [34] K. C. Foo, Z. M. Jiang, B. Adams, A. E. Hassan, Y. Zou, and P. Flora, “An industrial case study on the automated detection of performance regressions in heterogeneous environments,” in *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE ’15)*, IEEE, 2015, pp. 159–168. DOI: 10.1109/ICSE.2015.144.

- [35] W. Liu, Y. W. Heng, F. Lin, T.-H. Chen, and A. E. Hassan, *Mobileupreg: Identifying user-perceived performance regressions in mobile os versions*, 2025. arXiv: 2509.16864 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2509.16864>.
- [36] N. Balasubramanian, A. Balasubramanian, and A. Venkataramani, “Energy consumption in mobile phones: A measurement study and implications for network applications,” in *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement (IMC '09)*, New York, NY, USA: Association for Computing Machinery, 2009, pp. 280–293. DOI: 10.1145/1644893.1644927.
- [37] D. Li, Y. Lyu, J. Gui, and W. G. J. Halfond, “Automated energy optimization of HTTP requests for mobile applications,” in *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*, ACM, 2016, pp. 249–260. DOI: 10.1145/2884781.2884867.
- [38] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, and A. Milovidov, “ClickHouse - lightning fast analytics for everyone,” *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3731–3744, 2024. DOI: 10.14778/3685800.3685802.
- [39] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013. DOI: 10.1145/2408776.2408794.
- [40] Android Developers. “Android’s kotlin-first approach.” Accessed: 19 April 2026. (2026), [Online]. Available: <https://developer.android.com/kotlin/first>.
- [41] A. Blanck, *Cheddar: Hacker news client for android*, <https://github.com/adrianblancode/Cheddar>, Accessed: 2025-06-09, 2024.

- [42] Google Play Console. “Device catalog.” Accessed: 21 Jun 2026. (2026), [Online]. Available: <https://play.google.com/console/about/devicecatalog/>.